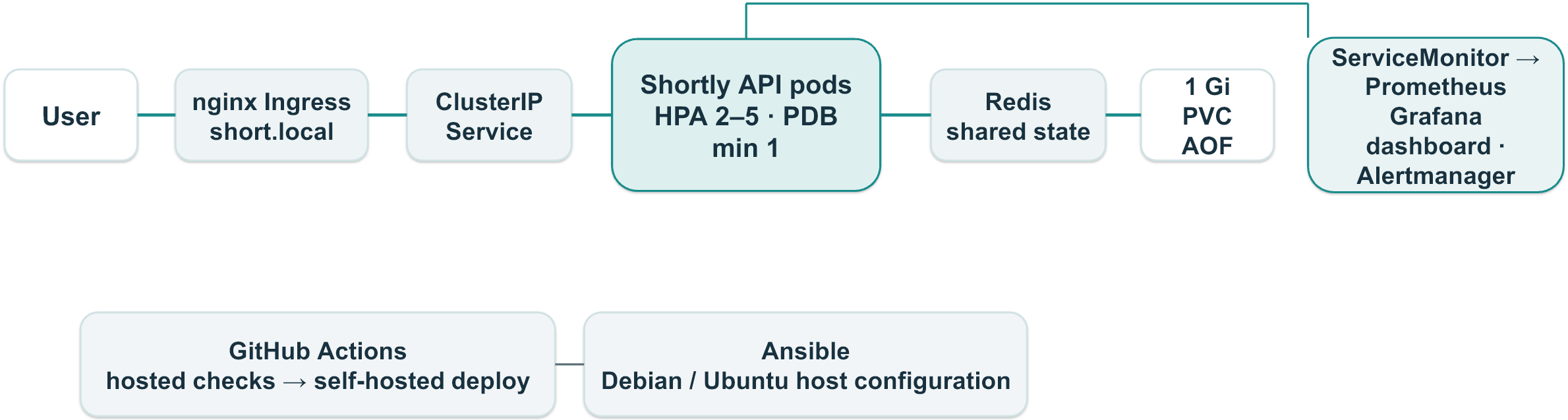
# Shortly: architecture and deployment

Shortly · FastAPI / Redis / Kubernetes / Prometheus / Grafana / Ansible



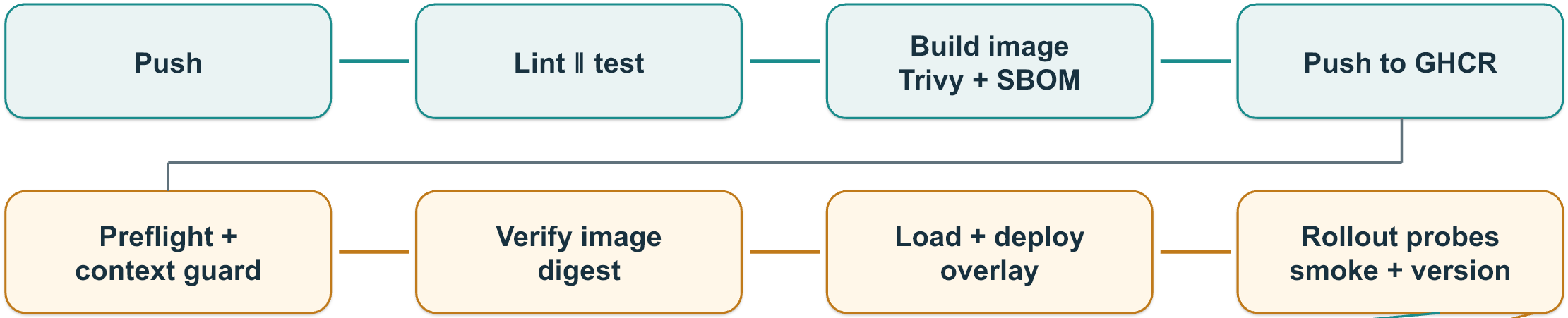

**Key choices: stateless API + Redis · readiness checks Redis; liveness does not maxUnavailable: 0 · one Redis replica in this local demo**

Metrics scrape the app Service; the local Alertmanager uses a null receiver.

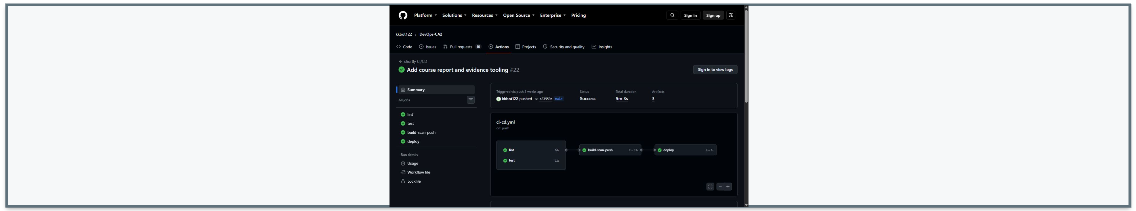
# GitHub Actions: build, scan, deploy and verify

Hosted checks protect builds; a guarded self-hosted runner updates Minikube.

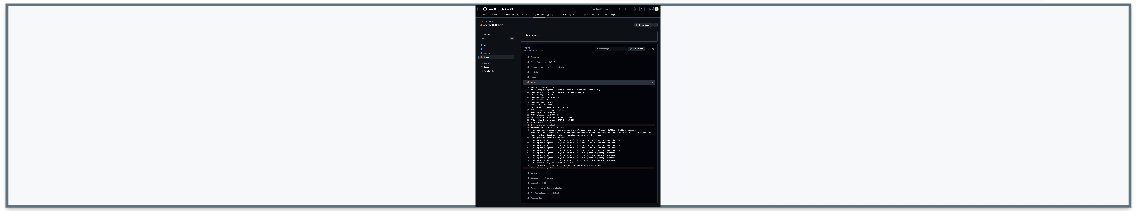
**GITHUB-HOSTED RUNNERS**




**Verified workflow run: 5 min 3 s (run 37209744711)**



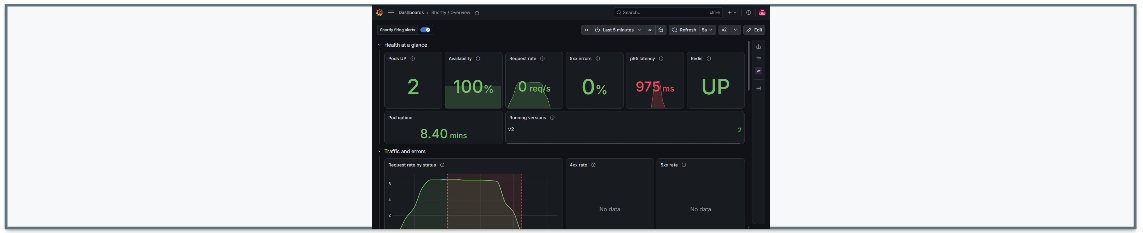
**Green workflow run**



**Failed release with automatic rollback**

# Evidence from deployments and failure scenarios

Fresh Windows checks; original records and full-size captures accompany this deck.




**Grafana: uptime, latency and error rate**




**Rolling update: v1 and v2 seen; 0 failures**




**CrashLoopBackOff observed, then rollback**




**Ansible second apply: changed=0**

**Fresh checks: 31 tests / 96.97% app coverage**
**240 rollout requests / 0 failures · Ansible changes: 31 then 0**

Full-size evidence: docs/screenshots/ · Original records: docs/evidence/windows/

# Challenges and fixes

Top evidence-backed challenges · details and sources: docs/CHALLENGES.md

**1 Ready probes missed release errors → health checks were too narrow → alert on 5xx and roll back**

2 Nested Kustomize base was blocked → path restriction → render a temporary sibling overlay

3 Grafana OOM at 256/384 MiB → limits were too low → use the evidenced 768 MiB setting

4 Redis outage broke readiness → API depends on Redis → separate liveness and verify recovery

5 Ansible changed on its second run → installed systemd binaries misled facts → check running systemd

# Lessons learned and next steps

Each lesson links to the challenge log; production work is future work.

**#1 Readiness and user success are different signals; alert on real request outcomes.**

#2 Render the exact manifest set before applying; layout affects deploy tooling.

#3 Resource limits need observed headroom; lower Grafana limits restarted the pod.

#4 Dependency-aware readiness protects traffic without restarting the API process.

#5 Check whether services are running; verify a second apply on a clean target.

**Next: use a multi-node cloud cluster; add SLO alerts, real alert routing and GitOps; isolate the self-hosted runner.**